

Description:

For the first lab project, we were assigned to make a hamming distance project coded in assembly language. Hamming distance is the difference in bits between two strings.

I decided to tackle this problem is that I looked for the smallest string and store its length. Using assembly language, I loop through the strings to the length of the shortest string and xor each bit to find where each bit differs. I added the number of differences in the bits and that will be the calculated hamming distance.

Code:

Custom Input:

```
        section .data
        str1: db "Enter the first string: ", 0
        len1 equ $ - str1

        str2: db "Enter the second string: ", 0
        len2 equ $ - str2

section .bss
        input_buffer1: resb 256 ; Reserve space for first input
        input_buffer2: resb 256 ; Reserve space for second input
        ham: resb 4           ; Reserve space for storing Hamming distance (4 bytes)

section .text
global main
main:
        ; Prompt for first string input
        mov eax, 1
        mov edi, 1
        mov esi, str1
        mov edx, len1
        syscall

        ; Read first input
        mov eax, 0
        mov edi, 0
        mov esi, input_buffer1
```

```
mov edx, 255
syscall
```

```
; Prompt for second string input
mov eax, 1
mov edi, 1
mov esi, str2
mov edx, len2
syscall
```

```
; Read second input
mov eax, 0
mov edi, 0
mov esi, input_buffer2
mov edx, 255
syscall
```

```
; Initialize registers for Hamming distance calculation
xor r9, r9      ; r9 will store Hamming distance count (set to 0)
xor r10, r10    ; r10 will be used to point to input_buffer1
xor r11, r11    ; r11 will be used to point to input_buffer2
xor r12, r12    ; r12 will be the loop counter
```

```
; Compare the strings byte by byte
; Compare until the shorter string is exhausted
mov r10, input_buffer1
mov r11, input_buffer2
```

```
compare_loop:
    mov al, [r10]    ; Load byte from first input string
    mov bl, [r11]    ; Load byte from second input string
    cmp al, bl       ; Compare the bytes
    jne different    ; If the bytes are different, jump to 'different'

    ; Bytes are equal, continue
    inc r10
    inc r11
    inc r12
    cmp r12, 256     ; Check if we've reached the end of input
    jl compare_loop
```

jmp done

different:

```
inc r9      ; Increment Hamming distance count
inc r10
inc r11
inc r12
cmp r12, 256
jl compare_loop
```

done:

```
; Store Hamming distance in ham buffer (as a number)
mov [ham], r9 ; Store Hamming distance in ham

; Print the Hamming distance
mov eax, 1
mov edi, 1
mov esi, ham
mov edx, 4 ; Print 4 bytes (Hamming distance as integer)
syscall
```

exit:

```
mov eax, 60 ; sys_exit
xor edi, edi ; Exit status 0
syscall
```

Foo/Bar: Test Case:

```
section .data
result_prompt db "Hamming Distance: ", 0
newline db 0xA, 0
foo db "foo", 0 ; Hardcoded string "foo"
bar db "bar", 0 ; Hardcoded string "bar"
```

section .bss

```
hamming_distance resd 1 ; Reserve 4 bytes for Hamming distance
distance_str resb 10 ; Buffer for the Hamming distance string
```

global main

section .text

main:

```
; Initialize Hamming distance to 0
mov dword [hamming_distance], 0
```

```
; Initialize the index for string comparison
xor ecx, ecx      ; Set ECX to 0 (index for string comparison)
```

```
compare_loop:
```

```
; Load the current byte of both strings
mov al, [foo + ecx]
mov bl, [bar + ecx]
```

```
; If both strings are null-terminated, exit the loop
cmp al, 0
je done
cmp bl, 0
je done
```

```
; Calculate the difference (XOR) between the characters (bitwise comparison)
xor al, bl        ; XOR the two characters to get the differing bits
```

```
; Count the number of 1 bits in the XOR result (Hamming distance for this byte)
mov ah, al        ; Copy the result into AH for bit counting
xor edx, edx      ; Clear EDX (Hamming distance accumulator)
```

```
count_bits:
```

```
test ah, 1        ; Test the least significant bit (LSB)
jz no_bit         ; If the bit is 0, jump to no_bit
inc edx           ; If the bit is 1, increment the counter
```

```
no_bit:
```

```
shr ah, 1         ; Shift the bits to the right
jnz count_bits    ; Repeat for all bits until AH is 0
```

```
; Add the Hamming distance for this byte to the total
add [hamming_distance], edx
```

```
; Move to the next byte in the strings
inc ecx
jmp compare_loop
```

```
done:
```

```
; Print the result prompt: "Hamming Distance: "
```

```
mov eax, 1      ; syscall number for write
mov edi, 1      ; file descriptor 1 (stdout)
mov esi, result_prompt ; pointer to the result prompt
mov edx, 18     ; length of result prompt
syscall
```

```
; Print the Hamming distance as a number (converting it to string)
mov eax, [hamming_distance] ; Load the Hamming distance into eax
mov ecx, 10      ; Base 10 for number conversion
mov ebx, distance_str ; Buffer for converted digits
add ebx, 10      ; Move the buffer pointer to the end
mov byte [ebx], 0 ; Null-terminate the buffer
```

```
; Convert to string (reverse order)
```

```
convert_to_str:
```

```
xor edx, edx    ; Clear edx (remainder)
div ecx         ; Divide eax by 10, quotient in eax, remainder in edx
dec ebx         ; Move the pointer back for the next character
add dl, '0'     ; Convert remainder to ASCII character
mov [ebx], dl   ; Store character in buffer
test eax, eax   ; Check if quotient is zero
jnz convert_to_str ; If not, continue dividing
```

```
; Move the buffer pointer back to the start of the string
mov esi, ebx    ; Set esi to point to the beginning of the string
```

```
; Print the Hamming distance string from the correct position
```

```
print_loop:
```

```
mov al, [esi]   ; Load character
test al, al     ; Check if it is null terminator
jz done_printing ; If null terminator, stop printing
mov eax, 1      ; syscall number for write
mov edi, 1      ; file descriptor 1 (stdout)
mov edx, 1      ; length of character
syscall
inc esi         ; Move to the next character
jmp print_loop
```

```
done_printing:
```

```
; Print newline
```

```

mov eax, 1      ; syscall number for write
mov edi, 1      ; file descriptor 1 (stdout)
mov esi, newline ; pointer to the newline character
mov edx, 1      ; length of newline
syscall

; Exit the program
mov eax, 60     ; syscall number for exit
xor edi, edi     ; status 0
syscall

```

Output:

The output for the foo/bar specific code is 8, however I was not able to figure out how to code the custom output.

For Foo/Bar:

```

[aamuah1@linux6 Hamming_Distance]$ nasm -g -f elf64 -l hamming.lst hamming.asm
[aamuah1@linux6 Hamming_Distance]$ gcc -m64 -o hamming hamming.o
[aamuah1@linux6 Hamming_Distance]$ ./hamming
Hamming Distance: 8

```